CÓDIGOCLARO · RUTA KOTLIN

Tema 12 · Genéricos

Guía de estudio · DAM/DAW

Tipos parametrizados, variance y restricciones.

1. Explicación

Un parámetro de tipo como T permite que una función opere con distintos tipos y conserve el tipo concreto en el retorno. La inferencia evita escribir argumentos de tipo cuando se pueden deducir. Kotlin usa varianza declarada en el tipo: out marca un productor que entrega T y permite una relación covariante; in marca un consumidor que acepta T y permite contravarianza. Las restricciones where acotan qué operaciones puede utilizarse sobre T.

2. Ejemplo

Código · kotlin

```kotlin
fun <T> primeroSeguro(valores: List<T>): T? = valores.firstOrNull()

fun main() {
    println(primeroSeguro(listOf("a", "b")))
    println(primeroSeguro(emptyList<Int>()))
}
```

Resultado esperado

```
a
null
```

3. Errores frecuentes

- Usar Any para evitar el parámetro de tipo y luego forzar casts.
- Confundir nullabilidad T? con el parámetro de tipo T.
- Declarar out en un tipo que también consume valores de T.

4. Ejercicio

Implementa una función genérica que devuelva el último valor o null si la lista está vacía.

Casos que debe superar

Entrada	Resultado esperado
[]	null
[7]	7
["a", "b"]	"b"

5. Solución explicada

Código · kotlin

```kotlin
fun <T> ultimoSeguro(valores: List<T>): T? = valores.lastOrNull()
```

Criterio de aceptación: Se acepta una implementación manual si mantiene T sin convertir los valores a Any ni hacer casts inseguros.

Otras soluciones válidas

- Usar firstOrNull para obtener el último valor equivalente con lastOrNull.
- Recorrer la lista guardando el último elemento y conservar el tipo T.

6. Fuentes oficiales

JetBrains / Kotlin — Generics: in, out, where